WEEK 7 · PHASE 3

Joins & Aggregations

All Join Types · Dedup · count/approx · summary()

- All join types: inner, left, right, full, cross, semi, anti
- Broadcast join hint for small tables
- Multi-key & complex join conditions
- Deduplication: dropDuplicates, window ROW_NUMBER
- Aggregations: count, approx_count_distinct, groupBy
- Practice Questions (10 Qs)

Table of Contents

1. Join Types Overview 3
2. Inner & Outer Joins 4
3. Semi, Anti & Cross Joins 6
4. Broadcast Join 8
5. Multi-Key & Complex Joins 9
6. Deduplication 11
7. Aggregations 13
8. union & unionByName 15
9. Practice Questions 16
10. Answer Key 19

WEEK 7 **Join Types Overview** All join types with behavior

1. Join Types Overview

Join Type	Returns	Null in result	PySpark how=
INNER	Rows matching in BOTH tables	Neither side	inner (default)
LEFT OUTER	All left rows + matching right	Right side when no match	left or left_outer
RIGHT OUTER	All right rows + matching left	Left side when no match	right or right_outer
FULL OUTER	All rows from both tables	Both sides when no match	outer or full or full_outer
LEFT SEMI	Left rows that HAVE a match (no right cols)	None	left_semi
LEFT ANTI	Left rows that DO NOT have a match	None	left_anti
CROSS	Cartesian product (every row × every row)	None	cross

■ **Exam Focus** Exam tip: LEFT SEMI is like INNER but returns only left columns. LEFT ANTI is the opposite of LEFT SEMI — returns rows with NO match. Neither includes right-side columns.

2. Inner & Outer Joins

2.1 INNER JOIN

■ INNER JOIN

```
# PySpark
df_orders.join(df_customers, on="customer_id", how="inner")
# or explicit condition
df_orders.join(df_customers,
  df_orders["customer_id"] == df_customers["customer_id"],
  how="inner")

-- SQL
SELECT o.*, c.name FROM silver.orders o
INNER JOIN silver.customers c ON o.customer_id = c.customer_id;
```

2.2 LEFT OUTER JOIN

■ LEFT JOIN

```
# Returns ALL orders; customers.name = null if no match
df_orders.join(df_customers, on="customer_id", how="left")

-- SQL
SELECT o.order_id, c.name
FROM silver.orders o
LEFT JOIN silver.customers c ON o.customer_id = c.customer_id;
```

2.3 FULL OUTER JOIN

■ FULL OUTER JOIN

```
# Returns ALL rows from both — nulls on whichever side has no match
df_a.join(df_b, on="id", how="outer")

-- SQL
SELECT * FROM table_a FULL OUTER JOIN table_b ON a.id = b.id;
```

2.4 Resolving Ambiguous Columns After Join

■ Ambiguous column resolution

```
# Problem: both DataFrames have "customer_id" → ambiguity
joined = df_orders.join(df_customers, on="customer_id")
# "on" string form automatically deduplicates the join key

# If using condition form — use aliases to disambiguate
orders = df_orders.alias("o")
customers = df_customers.alias("c")
joined = orders.join(customers, col("o.customer_id") == col("c.customer_id"))
joined.select("o.customer_id", "o.amount", "c.name")
```

3. Semi, Anti & Cross Joins

3.1 LEFT SEMI JOIN — Filter by Existence

■ LEFT SEMI JOIN

```
# Return orders that HAVE a matching customer
# Only order columns returned — no customer columns
df_orders.join(df_customers, on="customer_id", how="left_semi")

-- SQL equivalent using IN / EXISTS
SELECT * FROM silver.orders
WHERE customer_id IN (SELECT customer_id FROM silver.customers);
```

3.2 LEFT ANTI JOIN — Filter by Non-Existence

■ LEFT ANTI JOIN

```
# Return orders that have NO matching customer (orphaned orders)
df_orders.join(df_customers, on="customer_id", how="left_anti")

-- SQL equivalent using NOT IN / NOT EXISTS
SELECT * FROM silver.orders
WHERE customer_id NOT IN (SELECT customer_id FROM silver.customers);
```

3.3 CROSS JOIN — Cartesian Product

■ CROSS JOIN

```
# Every row in left × every row in right
# WARNING: extremely expensive! 1000 × 1000 = 1,000,000 rows
df_dates.join(df_products, how="cross")

-- SQL
SELECT * FROM date_dim CROSS JOIN product_dim;
-- Use case: generate all date-product combinations for a report
```

■■ **Warning** CROSS JOIN with large tables causes OOM errors. Only use for small lookup tables (e.g., date ranges × small dimension). Spark requires explicit how="cross" to prevent accidents.

4. Broadcast Join

Broadcast join sends a copy of a small DataFrame to every executor, eliminating the expensive shuffle required by sort-merge join. It dramatically improves performance when one side is small.

■ Broadcast join hint

```
from pyspark.sql.functions import broadcast

# Explicit broadcast hint — forces broadcast of df_country
df_orders.join(broadcast(df_country), on="country_code", how="left")

-- SQL hint
SELECT /*+ BROADCAST(c) */ o.*, c.country_name
FROM silver.orders o
JOIN silver.country c ON o.country_code = c.country_code;
```

Aspect	Sort-Merge Join (default)	Broadcast Join
Mechanism	Shuffle both sides, sort, merge	Send small table to all executors
When used	Both tables large	One table small (< autoBroadcastJoinThreshold)
Performance	Slower (shuffle = expensive)	Much faster (no shuffle)
Memory risk	Low	OOM if broadcast table too large
Config	Default for large joins	spark.sql.autoBroadcastJoinThreshold = 10MB default

■ Broadcast threshold config

```
# Adjust auto-broadcast threshold (default: 10MB)
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "50MB")

# Disable auto-broadcast
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")
```

■ **Exam Focus** Databricks auto-broadcasts tables < autoBroadcastJoinThreshold (default 10MB). For slightly larger small tables (10–50MB), use explicit broadcast() hint.

5. Multi-Key & Complex Join Conditions

■ Multi-key and non-equi joins

```
# Multi-key join using list
df_a.join(df_b, on=["country_code","product_id"], how="inner")

# Multi-key with explicit conditions
df_a.join(df_b,
  (df_a["country_code"] == df_b["country_code"]) &
  (df_a["product_id"]   == df_b["product_id"]),
  how="inner")

# Non-equi join (range condition)
df_events.join(df_windows,
  (df_events["ts"] >= df_windows["start_ts"]) &
  (df_events["ts"] <  df_windows["end_ts"]),
  how="inner")
```

6. Deduplication

Duplicate rows are common in Bronze tables due to retry logic, multiple source feeds, or CDC events. Silver tables must deduplicate before loading.

6.1 dropDuplicates() — Simple Dedup

■ **dropDuplicates() variants**

```
# Remove fully duplicate rows
df.dropDuplicates()

# Dedup based on key columns (keep first occurrence)
df.dropDuplicates(["order_id"])

# Dedup on composite key
df.dropDuplicates(["customer_id", "event_date", "event_type"])

# distinct() — alias for full row dedup
df.distinct()
```

■ **Exam Focus** dropDuplicates() keeps an arbitrary row when there are duplicates — it does not guarantee "latest" record. For "keep latest" logic, use Window + ROW_NUMBER().

6.2 Window ROW_NUMBER() — Keep Latest Record

■ **Window ROW_NUMBER() deduplication**

```
from pyspark.sql.functions import row_number, col
from pyspark.sql.window import Window

# Assign row number within each order_id partition, newest first
w = Window.partitionBy("order_id").orderBy(col("updated_at").desc())

df_deduped = (
  df.withColumn("rn", row_number().over(w))
    .filter(col("rn") == 1)
    .drop("rn")
)

-- SQL equivalent
SELECT * EXCEPT(rn) FROM (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY order_id ORDER BY updated_at DESC) AS rn
  FROM bronze.orders
) WHERE rn = 1;
```

6.3 Dedup Comparison

Method	Keeps	Performance	When to Use
dropDuplicates()	Arbitrary row	Fast (hash-based)	Exact duplicates; any row is fine
dropDuplicates(keys)	Arbitrary row per key	Fast	Key-based dedup; order doesn't matter
Window ROW_NUMBER()	Deterministic (e.g., latest)	Slower (window function)	Must keep specific row (newest/oldest)
MERGE INTO (upsert)	Latest from source	Medium	Ongoing incremental dedup at write time

7. Aggregations

7.1 Core Aggregate Functions

■ groupBy + agg

```
from pyspark.sql.functions import count, countDistinct, approx_count_distinct,
    sum, avg, min, max, mean, stddev, collect_list, collect_set

df.groupBy("region").agg(
  count("order_id").alias("total_orders"),
  countDistinct("customer_id").alias("unique_customers"),
  approx_count_distinct("customer_id", 0.05).alias("approx_customers"),
  sum("amount").alias("total_revenue"),
  avg("amount").alias("avg_order_value"),
  min("order_date").alias("first_order"),
  max("order_date").alias("last_order"),
)
```

7.2 approx_count_distinct vs countDistinct

Function	Accuracy	Performance	Use Case
countDistinct()	Exact	Slow (full shuffle)	Small-medium datasets, exact count required
approx_count_distinct(col, rsd)	~rsd% error (default 5%)	Very fast (HyperLogLog)	Large datasets, dashboards, approximate OK

■ approx_count_distinct rsd parameter

```
# rsd = relative standard deviation (0.05 = ~5% error)
approx_count_distinct("user_id", 0.05)   # fast, ~5% error
approx_count_distinct("user_id", 0.01)   # more accurate, slower
```

7.3 summary() and describe()

■ describe() and summary()

```
# describe(): count, mean, stddev, min, max for numeric/string cols
df.describe("amount", "quantity").show()

# summary(): same as describe + percentiles
df.summary("count","mean","stddev","min","25%","50%","75%","max").show()

# SQL equivalent for quick stats
-- SELECT count(*), avg(amount), min(amount), max(amount)
-- FROM silver.orders;
```

7.4 Window Aggregations (Running Totals)

■ Window aggregations

```
from pyspark.sql.functions import sum, avg, rank
from pyspark.sql.window import Window

# Running total by customer
w = Window.partitionBy("customer_id").orderBy("order_date")
         .rowsBetween(Window.unboundedPreceding, Window.currentRow)

df.withColumn("running_total", sum("amount").over(w))

# Rank within region
w2 = Window.partitionBy("region").orderBy(col("amount").desc())
df.withColumn("rank", rank().over(w2))
```

8. union & unionByName

■ **union vs unionByName**

```
# union(): stacks rows — columns matched by POSITION (not name)
# WARNING: schemas must match exactly in order
df_q1.union(df_q2)

# unionByName(): matched by COLUMN NAME (safe for different order)
df_jan.unionByName(df_feb)

# unionByName with extra columns (allowMissingColumns)
df_new.unionByName(df_old, allowMissingColumns=True)
# Missing columns filled with null

-- SQL equivalent
-- SELECT * FROM q1 UNION ALL SELECT * FROM q2;
-- UNION removes duplicates; UNION ALL keeps all (faster)
```

■ **Exam Focus** Always prefer unionByName over union() to avoid subtle bugs when column order differs between DataFrames. UNION ALL is faster than UNION (no dedup step).

QUIZ **Practice Questions** Week 7 — Joins, Dedup & Aggregations

9. Practice Questions

1. A query joins a 500GB orders table with a 5MB country lookup table. Which join strategy provides the best performance?

A) Sort-merge join (default)

B) Broadcast join — broadcast the small country table

C) Hash join on country_code

D) Left semi join

2. Which join type returns ONLY the left-side columns and only for rows that HAVE a match in the right table?

A) INNER JOIN

B) LEFT OUTER JOIN

C) LEFT SEMI JOIN

D) LEFT ANTI JOIN

3. df_a.dropDuplicates(["order_id"]) keeps which row when multiple rows share the same order_id?

A) The row with the latest updated_at

B) The row with the smallest order_id

C) An arbitrary row — no guaranteed order

D) The first row in file order

4. A dashboard needs approximate distinct user counts from a 10-billion-row table. Which function is most appropriate?

A) countDistinct("user_id")

B) count("user_id")

C) approx_count_distinct("user_id", 0.05)

D) collect_set("user_id").size()

5. Which join type returns rows from the left table that have NO matching row in the right table?

A) LEFT OUTER JOIN where right side is null

B) LEFT ANTI JOIN

C) INNER JOIN with null filter

D) NOT LEFT SEMI JOIN

6. A developer uses df_a.union(df_b) but the DataFrames have columns in different orders. What is the result?

A) Spark automatically matches by column name

B) Columns are matched by position — wrong data in wrong columns

C) Spark raises an AnalysisException

D) The result has duplicate columns

7. Which Window function assigns a unique, sequential number to rows within each partition ordered by a column?

A) rank()

B) dense_rank()

C) row_number()

D) ntile()

8. A CROSS JOIN between a 1000-row date table and a 500-row product table produces how many rows?

A) 500

B) 1000

C) 1500

D) 500,000

9. To adjust auto-broadcast threshold from 10MB to 50MB, which config should be set?

A) spark.sql.broadcastTimeout = "50MB"

B) spark.sql.autoBroadcastJoinThreshold = "52428800" # bytes

C) spark.broadcast.maxSize = "50MB"

D) spark.sql.joinStrategy = "broadcast:50MB"

10. An analyst uses df.summary(). Which statistics does summary() return that describe() does NOT?

A) mean and stddev

B) min and max

C) Percentiles (25%, 50%, 75%)

D) count

ANSWERS **Answer Key** Week 7 Practice Questions

10. Answer Key

#	Ans	Explanation
1	(B)	Broadcast join sends the small table (5MB) to all executors, eliminating the expensive shuffle of the 500GB orders table. Databricks auto-broadcasts if < 10MB, or use explicit broadcast() hint.
2	(C)	LEFT SEMI JOIN returns left-side rows that have at least one match in the right side, but includes ONLY left-side columns. It is equivalent to WHERE EXISTS.
3	(C)	dropDuplicates() does not guarantee which row is kept — it keeps an arbitrary one. For deterministic deduplication (e.g., "keep latest"), use Window ROW_NUMBER() with an explicit ORDER BY.
4	(C)	approx_count_distinct() uses HyperLogLog algorithm for near-exact distinct counts with minimal memory. It is orders of magnitude faster than countDistinct() on large datasets.
5	(B)	LEFT ANTI JOIN (how="left_anti") returns only left-side rows with no match in the right. It is equivalent to WHERE NOT EXISTS or WHERE key NOT IN (...).
6	(B)	union() matches columns by POSITION, not name. If column order differs, data lands in the wrong columns silently. Always use unionByName() to match by column name.
7	(C)	row_number() assigns unique sequential numbers (1, 2, 3...) within each partition. rank() and dense_rank() can produce ties; row_number() never does.
8	(D)	CROSS JOIN is a Cartesian product: 1000 × 500 = 500,000 rows. Every row from the left table is combined with every row from the right table.
9	(B)	spark.sql.autoBroadcastJoinThreshold controls the threshold in BYTES. 50MB = 52,428,800 bytes. The string "50MB" notation also works in Spark 3.x+.
10	(C)	summary() includes all describe() statistics (count, mean, stddev, min, max) PLUS percentiles (25%, 50%, 75%). describe() does not include percentiles.

11. Week 7 Cheat Sheet

Join Type	how= value	Returns
Inner	inner	Matching rows only
Left Outer	left / left_outer	All left + matching right
Right Outer	right / right_outer	All right + matching left
Full Outer	outer / full	All rows both sides
Left Semi	left_semi	Left rows WITH match (no right cols)
Left Anti	left_anti	Left rows WITHOUT match
Cross	cross	Cartesian product (ALL × ALL)

Agg Function	Description	Notes
count("col")	Count non-null values	Fast
countDistinct("col")	Exact distinct count	Slow on large data
approx_count_distinct(col,rsd)	Approximate distinct count (HLL)	Very fast; default rsd=0.05
sum/avg/min/max	Standard aggregates	Standard SQL
collect_list()	Array of all values (with dups)	Use carefully — can cause OOM
collect_set()	Array of distinct values	Smaller than collect_list
row_number()	Unique seq per partition	Used for dedup "keep latest"

BONUS **Advanced Joins & Window Lab** Deep Dive & Exam Scenarios

A. Window Functions Complete Reference

Window Function	Description	Returns Ties As
row_number()	Unique sequential number — no ties	Always unique (1,2,3,4)
rank()	Rank with gaps on tie	1,1,3,4 (gap at 2)
dense_rank()	Rank without gaps on tie	1,1,2,3 (no gap)
lag(col,n)	Value from N rows before current	null for first N rows
lead(col,n)	Value from N rows after current	null for last N rows
first_value(col)	First value in window frame	First row in partition
last_value(col)	Last value in window frame	Last row in partition
sum/avg/min/max(col).over(w)	Running aggregation	Cumulative value
ntile(n)	Divide partition into N buckets	Bucket number 1..N

■ Window functions examples

```
from pyspark.sql.functions import lag, lead, rank, dense_rank, sum, first_value
from pyspark.sql.window import Window

w = Window.partitionBy("customer_id").orderBy("order_date")

# Previous order amount (lag)
df.withColumn("prev_amount", lag("amount", 1).over(w))

# Running total
w_cum = w.rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("running_total", sum("amount").over(w_cum))

# Rank customers by revenue within region
w_rank = Window.partitionBy("region").orderBy(col("total_revenue").desc())
df.withColumn("revenue_rank", rank().over(w_rank))
```

B. Join Performance & Skew

B.1 Data Skew in Joins

Data skew occurs when one join key value has far more rows than others (e.g., 90% of orders from one customer). This causes one executor to process most data while others are idle.

■ Skew handling with salting

```
# Symptom: one Spark task takes 100x longer than others
# Check in Spark UI: Stages → Task Metrics → Duration

# Solution 1: Salt the key (add random suffix)
from pyspark.sql.functions import rand, floor, concat, lit
N = 10  # number of salt buckets
df_large = df_large.withColumn("salt", (floor(rand() * N)).cast("int"))
  .withColumn("salted_key", concat(col("customer_id"), lit("_"), col("salt")))

# Explode the small table to match all salts
from pyspark.sql.functions import array, explode
df_small = df_small.withColumn("salt", array([lit(i) for i in range(N)]))
  .withColumn("salt", explode("salt"))
  .withColumn("salted_key", concat(col("customer_id"), lit("_"), col("salt")))

# Join on salted key
df_result = df_large.join(df_small, on="salted_key").drop("salt","salted_key")
```

■ **Tip** AQE (Adaptive Query Execution) handles most skew automatically via spark.sql.adaptive.skewJoin.enabled=true (default). Manual salting is needed for extreme skew cases.

C. Deduplication Lab — Complete Pattern

■ Full deduplication pipeline

```
# Full deduplication pipeline: Bronze → Silver
from pyspark.sql.functions import row_number, col, current_timestamp
from pyspark.sql.window import Window

df_bronze = spark.table("catalog.bronze.orders")

# Step 1: Window to rank duplicates
w = Window.partitionBy("order_id").orderBy(col("event_ts").desc())

# Step 2: Assign rank — 1 = latest record
df_ranked = df_bronze.withColumn("_rn", row_number().over(w))

# Step 3: Keep only rank 1
df_deduped = df_ranked.filter(col("_rn") == 1).drop("_rn")

# Step 4: MERGE into silver (upsert to handle reprocessing)
from delta.tables import DeltaTable
silver = DeltaTable.forName(spark, "catalog.silver.orders")
silver.alias("t").merge(df_deduped.alias("s"), "t.order_id = s.order_id")
  .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()

print(f"Deduped: {df_deduped.count()} unique orders")
```

D. Self-Assessment Checklist

- ■ I know all 7 join types and their how= values
- ■ I understand LEFT SEMI (filter by existence, left cols only) vs INNER (both cols)
- ■ I understand LEFT ANTI (filter by non-existence) vs NOT EXISTS
- ■ I know when to use broadcast() hint vs relying on autoBroadcastJoinThreshold
- ■ I can write a CROSS JOIN and understand its Cartesian product risk
- ■ I know dropDuplicates() is non-deterministic for "keep latest"
- ■ I can write Window ROW_NUMBER() deduplication with desc() ordering
- ■ I know approx_count_distinct rsd parameter and when to use it vs countDistinct
- ■ I know union() matches by position; unionByName() matches by name
- ■ I can write Window running totals and rank functions

■ **Exam Focus** Week 7 covers critical content for Domain 3 (22%). Joins + dedup + aggregations appear in 4-5 exam questions. Master the join type matrix and dropDuplicates vs ROW_NUMBER distinction.

E. Aggregation Deep Dive

E.1 groupBy with Multiple Aggregations

```
from pyspark.sql.functions import (
  count, countDistinct, approx_count_distinct, sum, avg, min, max,
  first, last, collect_list, collect_set, variance, stddev
)

df.groupBy("region","product_category").agg(
  count("*").alias("total_rows"),
  count("order_id").alias("orders_with_id"),  # excludes nulls
  countDistinct("customer_id").alias("unique_customers"),
  approx_count_distinct("session_id",0.05).alias("approx_sessions"),
  sum("revenue").alias("total_revenue"),
  avg("revenue").alias("avg_revenue"),
  min("order_date").alias("first_order"),
  max("order_date").alias("last_order"),
  stddev("revenue").alias("revenue_stddev"),
  collect_list("sku").alias("all_skus"),     # list with duplicates
  collect_set("sku").alias("unique_skus"),   # set — no duplicates
)
```

E.2 HAVING — Filter After Group

```
-- SQL HAVING = filter on aggregated result
SELECT region, SUM(revenue) AS total_revenue
FROM silver.orders
GROUP BY region
HAVING SUM(revenue) > 1000000;

# PySpark equivalent
from pyspark.sql.functions import sum as spark_sum
df.groupBy("region")
  .agg(spark_sum("revenue").alias("total_revenue"))
  .filter(col("total_revenue") > 1000000)
```

F. Join + Dedup Exam Scenarios

Scenario	Best Approach
"Find customers with at least one order"	LEFT SEMI JOIN orders on customer_id
"Find customers with NO orders (orphaned)"	LEFT ANTI JOIN orders on customer_id
"Combine 2 tables, all rows from both sides"	FULL OUTER JOIN
"Small 5MB lookup table join"	broadcast(small_df) hint
"Keep only latest row per order_id"	Window ROW_NUMBER() ORDER BY updated_at DESC, filter rn=1
"Remove exact duplicate rows"	df.distinct() or dropDuplicates()
"Stack two DataFrames with different column orders"	unionByName() — NOT union()
"Approximate distinct users for dashboard"	approx_count_distinct("user_id", 0.05)
"All date × product combinations"	CROSS JOIN (use only for small tables!)

G. Week 7 Key Facts Summary

- INNER = matching rows only | LEFT = all left | RIGHT = all right | FULL = all rows
- LEFT SEMI: left rows WITH match, left cols only (like WHERE EXISTS)
- LEFT ANTI: left rows WITHOUT match (like WHERE NOT EXISTS)
- CROSS JOIN = N × M rows — dangerous on large tables
- union() by POSITION — wrong if column order differs between DFs
- unionByName() by NAME — safe for mismatched column orders
- allowMissingColumns=True fills missing columns with null
- dropDuplicates(keys) is non-deterministic — use ROW_NUMBER() for "keep latest"
- approx_count_distinct uses HyperLogLog — rsd=0.05 means ~5% error
- summary() includes percentiles (25%,50%,75%); describe() does not

H. Window Function Frame Reference

Frame Spec	Meaning	Common Use
rowsBetween(unboundedPreceding, currentRow)	All rows from start to current	Running total
rowsBetween(unboundedPreceding, unboundedFollowing)	All rows in partition	Partition-level agg over each row
rowsBetween(-2, 0)	Current row + 2 preceding	3-row moving average
rangeBetween(-6,0)	Value-based range (not row-based)	Date range windows
No frame spec (default)	Unbounded preceding to current row	Standard window agg

■■ **Warning** Window functions require Window.partitionBy() — without it, the entire DataFrame is one partition (all data sent to one executor = OOM risk on large tables).